

A proof of Conjecture 00000000206

Every value occurs only finitely often in Recamán's sequence

1 The conjecture

Conjecture 00000000206 reads:

Definition: The Recamán sequence is defined by the backtracking-doubling recurrence.

Conjecture: Every nonnegative integer occurs only finitely often in the sequence; and the finiteness is settled by interval covering.

Reading. The Recamán sequence is OEIS A005132. The phrase “backtracking-doubling” is loose wording: no doubling occurs in the standard sequence, and we use the standard definition below. The first clause is the mathematical claim: for every integer $m \geq 0$, the set $\{n : a_n = m\}$ is finite. The second clause, “the finiteness is settled by interval covering” (Chinese text: the core of the finiteness is interval covering), names no object or property. We read it as a remark on the method of proof, not as a mathematical statement, and we do not formalize it. The proof below is an interval argument: every add-step occurrence of m lies at an index in $[1, m]$.

2 Definition

The retrieved Wikipedia article on Recamán's sequence (https://en.wikipedia.org/wiki/Recam%C3%A1n%27s_sequence) defines $a_0 = 0$ and, for $n \geq 1$,

$$a_n = \begin{cases} a_{n-1} - n & \text{if } a_{n-1} - n > 0 \text{ and is not already in the sequence,} \\ a_{n-1} + n & \text{otherwise.} \end{cases}$$

OEIS A005132 (<https://oeis.org/A005132>) states the same rule with “nonnegative” in place of “ > 0 ”. “Not already in the sequence” means not among $a_0, \dots, a_{n-1}$. The two guards give the same sequence, because the value $0 = a_0$ is always already present.

In Lean (ASCII rendering; indices shifted to $n + 1$, values in $\mathbb{N}$, so the subtraction is the true difference under the guard):

```
def IsRecaman (a : Nat -> Nat) : Prop :=
  a 0 = 0 /\ forall n, a (n + 1) =
    if n + 1 < a n /\ forall k <= n, a k != a n - (n + 1)
    then a n - (n + 1) else a n + (n + 1)
```

IsRecamanOEIS is the same with $n + 1 \leq a n$. The sequence `recaman` is built by list recursion (`recList n = [an, ..., a0]`). We prove `isRecaman_recaman`, `isRecamanOEIS_recaman` (it satisfies both forms) and `isRecaman_unique` (the recurrence determines the sequence). The first 27 terms are checked by `decide` against the Wikipedia list 0, 1, 3, 6, 2, 7, 13, 20, 12, 21, 11, 22, 10, 23, 9, 24, 8, 25, 43, 62, 42, 63, 41, 18, 42, 17, 43; note $a_{20} = a_{24} = 42$, so values can repeat.

3 Proof

Lemma 1 (`finite_of_step`). *Let $a : \mathbb{N} \rightarrow \mathbb{N}$ be such that for every n , either $a_{n+1} = a_n + (n + 1)$ or $a_{n+1} \neq a_k$ for all $k \leq n$. Then for every m the set $\{n : a_n = m\}$ is finite.*

Proof. Let $n > m$ with $a_n = m$; then $n = p + 1$. If $a_n = a_p + n$ then $m \geq n$, a contradiction. So $a_n \neq a_k$ for all $k < n$, i.e. n is the first index with value m . Hence at most one index $N > m$ has $a_N = m$. If there is one, $\{n : a_n = m\} \subseteq [0, N]$; otherwise $\{n : a_n = m\} \subseteq [0, m]$. $\square$

Theorem 2 (`finite_occurrences_of_isRecaman`, `finite_occurrences_of_isRecamanOEIS`, `recaman_finite_occurrences`). *For every sequence satisfying either form of the recurrence, and in particular for `recaman`, every value $m \in \mathbb{N}$ occurs only finitely often.*

Proof. In the subtract branch the guard says $a_{n+1} = a_n - (n + 1)$ is not among $a_0, \dots, a_n$; in the other branch $a_{n+1} = a_n + (n + 1)$. So the lemma applies. $\square$

4 Lean formalization

File `lean/Conjecture206/Basic.lean` (namespace `Conjecture206`, 166 lines, only import `Mathlib`). Main theorem (ASCII rendering):

```
theorem recaman_finite_occurrences (m : Nat) : {n | recaman n = m}.Finite
```

Supporting results: `finite_of_step`, `finite_occurrences_of_isRecaman`, `finite_occurrences_of_isRecamanOEIS`, `recList_succ`, `mem_recList`, `isRecaman_recaman`, `isRecamanOEIS_recaman`, `isRecaman_unique`.

Axiom audit. `#print` axioms for `recaman_finite_occurrences`, `finite_occurrences_of_isRecamanOEIS` and `isRecamanOEIS_recaman` reports only `propext`, `Classical.choice`, `Quot.sound` (`isRecaman_unique`: only `propext`, `Quot.sound`). There is no `sorry` and no `native_decide`.